ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
UNIVERSIDAD

Lab 15: Automation

Report made by

Laura Daniela Rodríguez Sánchez
Sergio Andrés Bejarano Rodríguez

Escuela Colombiana de Ingeniería Julio Garavito
May 11, 2026

Hands-on lab

Requirements: Kali Linux, Python 3.10+, nmap, whois, dig, curl

The lab is structured as a pipeline: each part produces output that the next part consumes. By the end you will have a complete, multi-stage reconnaissance and analysis tool.

Part 1: From sequential to concurrent scanner

Start with this working sequential scanner:

```
import socket
import time

def scan_port(host: str, port: int, timeout: float = 1.0) -> bool:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        try:
            s.connect((host, port))
            return True
        except (socket.timeout, ConnectionRefusedError, OSError):
            return False

if __name__ == "__main__":
    host = "127.0.0.1"
    start = time.perf_counter()
    open_ports = [p for p in range(1, 1025) if scan_port(host, p)]
    elapsed = time.perf_counter() - start
    print(f"Open ports: {open_ports} ({elapsed:.1f}s)")
```

To simulate realistic network conditions where ports do not respond immediately, an iptables DROP rule was applied to the outbound traffic on ports 100–300:

```
sudo iptables -A OUTPUT -p tcp --dport 100:300 -j DROP
```

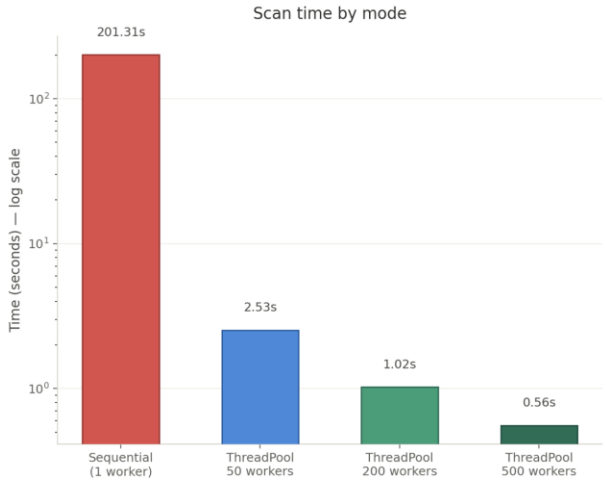
Then, execute:

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 scanner_v1.py
Open ports: [22, 80]
Time: 201.3129s
```

A. Run this against 127.0.0.1 (localhost) and record the elapsed time. Then rewrite it using ThreadPoolExecutor. Test max_workers values of 50, 200, and 500 and record the elapsed time for each. Plot or tabulate the results.

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 scanner_v2_threads.py
workers=  50 | Open ports: [22, 80] | Time: 2.5266s
workers= 200 | Open ports: [22, 80] | Time: 1.0229s
workers= 500 | Open ports: [22, 80] | Time: 0.5564s
```

The results:



B. Rewrite again using asyncio + asyncio.Semaphore to cap concurrency at a configurable limit. Measure its performance at the same concurrency levels and compare against the threaded version.

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 ~/lab-auto/scanner_v3_asyncio.py
limit=  50 | Open ports: [22, 80] | Time: 5.0266s
limit= 200 | Open ports: [22, 80] | Time: 2.0157s
limit= 500 | Open ports: [22, 80] | Time: 1.0160s
```

The results:

C. Add a proper CLI to whichever version you prefer, using argparse:

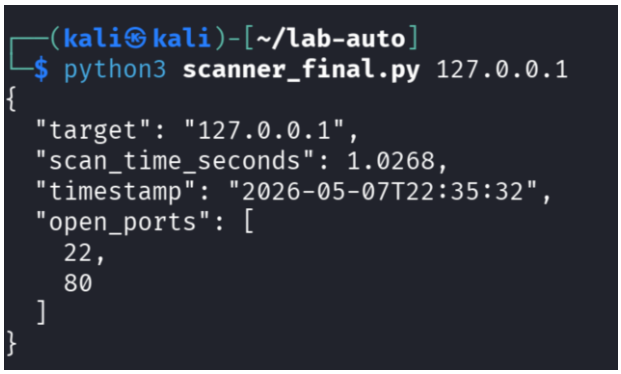
Flag	Description	Default
target	IP address to scan	(required)
--ports	Port range, e.g. 1-1024 or 22,80,443	1-1024
--rate	Max concurrent connections	200
--timeout	Per-port timeout in seconds	0.5
--output	JSON output file	stdout

Parse the --ports argument to support both comma-separated lists (22,80,443) and ranges (1-1024).

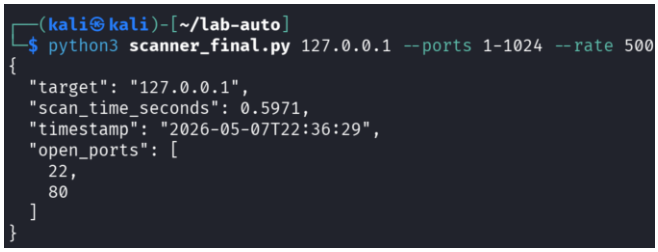
D. Output results as JSON to the path given by --output:

```
{
 "target": "127.0.0.1",
 "scan_time_seconds": 2.3,
 "timestamp": "2026-05-04T14:32:00",
 "open_ports": [22, 80, 443]
}
```

Basic use – stdout:



With port range and rate:



With a list of specific ports:

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 scanner_final.py 127.0.0.1 --ports 22,80,443,8080,8888
{
  "target": "127.0.0.1",
  "scan_time_seconds": 0.0018,
  "timestamp": "2026-05-07T22:38:53",
  "open_ports": [
    22,
    80,
    8080,
    8888
  ]
}
```

Save to json file:

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 scanner_final.py 127.0.0.1 --ports 1-1024 --output results.js
on
cat results.json
Results saved to results.json
{
  "target": "127.0.0.1",
  "scan_time_seconds": 1.0267,
  "timestamp": "2026-05-07T22:40:31",
  "open_ports": [
    22,
    80
  ]
}
```

Question

At very high concurrency (e.g. --rate 2000), you may observe false negatives (open ports reported as closed). Explain the mechanism behind this. Why does this mean “the scanner did not detect it” is not the same claim as “the port is closed”? What does this imply about how you should interpret scan results from any tool, including nmap?

Why false negatives occur at --rate 2000

The mechanism

When 2,000 threads fire simultaneously, the operating system must manage 2,000 concurrent TCP SYN packets originating from the same host. This exhausts several resources at once:

- **Ephemeral port exhaustion** — each outbound connection consumes a source port from the OS pool (typically 32,768–60,999 on Linux, ~28,000 ports). At --rate 2000 scanning 10,000 ports, the OS may run out of source ports mid-scan and silently drop new connection attempts.
- **Socket descriptor limits** — the process hits its open file descriptor limit (ulimit -n, default 1,024 on many systems). New socket() calls raise OSError which the scanner catches and treats as a closed port.

- **Kernel TCP backlog saturation** — the target's listening socket has a finite accept() backlog. Under extreme flood conditions, even 127.0.0.1 can drop incoming SYNs if the backlog queue fills before the application drains it.
- **Thread scheduler starvation** — 2,000 OS threads competing for CPU time cause context-switching overhead that delays some threads past their timeout window, even if the remote port would have responded in time.

In all four cases the connection fails on the **scanner side**, not because the target port is closed.

"Not detected" ≠ "port is closed"

A scanner can only report what it observed within its operating constraints. A false negative means the probe never reached the target, or the response never reached the scanner in time. The port state on the target is unchanged. The scanner's result is a statement about **the scan attempt**, not about the target. Concretely:

- A port behind a stateful firewall that silently drops SYNs looks identical to a false negative from resource exhaustion.
- A service that responds slowly (e.g. a rate-limited SSH daemon) may exceed the timeout and appear closed.
- A transient network hiccup during the scan window produces the same result as a genuinely closed port.

What this implies about interpreting any scan tool, including nmap

Every scan result carries implicit uncertainty that most reports omit. The correct framing is not *"port 443 is closed"* but *"port 443 did not respond to this probe, under these conditions, at this moment."* In practice this means:

- **Validate positives more than negatives.** An open port finding is reliable. A closed/filtered finding is only as trustworthy as the scan conditions.
- **Tune conservatively for completeness.** nmap's default --min-rate and timing templates (-T3) exist precisely to avoid false negatives caused by self-inflicted congestion. -T5 (insane) trades accuracy for speed the same way --rate 2000 does here.
- **Re-scan suspected ports individually.** If a host is believed to run a service on a specific port and the broad scan missed it, a single targeted probe with a longer timeout is more reliable than the bulk result.

- **Treat scan output as evidence, not ground truth.** In a penetration test report, findings are what was observed — absence of a finding is not proof of absence of the vulnerability.

Part 2: Structured output and enrichment

A. Run nmap against the lab network with service version detection and XML output:

```
nmap -sV --open -oX scan.xml 192.168.1.0/24
```

Before writing any code, open scan.xml and read through the structure. Identify how hosts, ports, states, service names, and version strings are organized in the XML hierarchy.

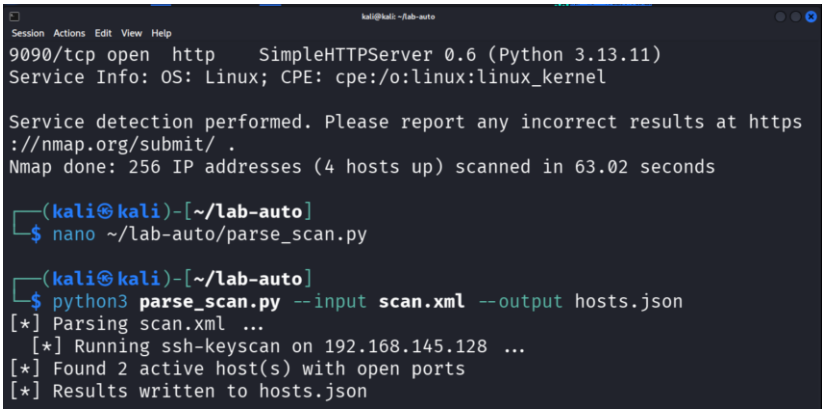
B. Write parse_scan.py that reads scan.xml and produces a structured Python object (dict) for each live host:

```
{
  "ip": "192.168.1.10",
  "hostname": "gateway.local",
  "open_ports": [
    {"port": 22, "service": "ssh", "version": "OpenSSH 8.9"},
    {"port": 80, "service": "http", "version": "Apache httpd 2.4.54"}
  ]
}
```

Use xml.etree.ElementTree — do not use a third-party nmap library.

C. For each host with port 22 open, run ssh-keyscan via subprocess and parse the output to extract the key type (e.g. ecdsa-sha2-nistp256, ssh-ed25519). Add a "ssh_host_key_type" field to that host's dict. Handle the case where ssh-keyscan times out or the host does not respond.

D. Write the enriched list of hosts to hosts.json. Your script should accept --input (XML file) and --output (JSON file) arguments.



Question

Service version detection (-sV) works by sending probe packets and matching responses against a signature database (nmap-service-probes). This makes the scan significantly slower and generates more network traffic than a plain port scan. From a defender's perspective: why is the version string in a service banner valuable intelligence for an attacker? What is the security-relevant difference between Apache httpd 2.4.54 and a server that returns no version string at all?

Why the version string is valuable intelligence for an attacker

When a server advertises Apache httpd 2.4.54, the attacker can immediately cross-reference that exact string against public vulnerability databases — CVE, ExploitDB, Metasploit's module list — and retrieve a precise list of known exploits for that version. The reconnaissance phase collapses

from hours of blind probing into a targeted lookup that takes seconds. Your own scan illustrates this directly: OpenSSH 10.2p1 Debian 3 tells an attacker the exact OpenSSH version, the packaging branch, and that the OS is Debian-based, all without sending a single additional packet beyond the -sV probe.

Version strings also reveal patch lag. A server running Apache httpd 2.4.54 when the current release is 2.4.66 signals that the organization has a slow patch cycle — a reliable predictor that other systems in the network are similarly outdated.

Apache httpd 2.4.54 vs. a server returning no version string

A server that suppresses its banner does not become invulnerable — if it runs a vulnerable version, that vulnerability still exists. The security-relevant difference is operational: it forces the attacker to move from passive reconnaissance to active fingerprinting. Instead of reading a banner, the attacker must send crafted probes, analyze behavioral differences in error responses, or attempt version-specific payloads — all of which generate network traffic that a defender can detect and alert on.

Banner suppression raises the cost and noise of reconnaissance. In Apache this is a two-line change:

```
ServerTokens Prod       # reports only "Apache", no version
ServerSignature Off     # removes version footer from error pages
```

The underlying principle is security through obscurity as a delay tactic, not a primary control. Suppressing the banner buys time and increases attacker visibility — it does not replace patching, but it removes the free intelligence that makes targeted exploitation trivially easy.

Part 3: Log analysis and anomaly detection

This part uses log files for analysis. Either use the real logs on your Kali system (/var/log/auth.log, if sshd is running) or generate a synthetic one with the script below. Save it as auth.log and access.log respectively.

Generate a synthetic auth log for testing:

```
python3 - <<'EOF'
import random, datetime
```

```
ips = ["10.0.0.1", "10.0.0.2", "185.220.101.5", "192.168.1.50", "45.33.32.156"]
users = ["root", "admin", "ubuntu", "daniel"]
now = datetime.datetime.now()

with open("auth.log", "w") as f:
    for _ in range(500):
        ip = random.choices(ips, weights=[2, 2, 40, 1, 30])[0]
        user = random.choice(users)
        ts = (now - datetime.timedelta(seconds=random.randint(0, 86400))).strftime("%b %d %H:%M:%S")
        f.write(f"{ts} kali sshd[1234]: Failed password for {user} from {ip} port {random.randint(40000,60000)} ssh2\n")
    # Add some successful logins
    for _ in range(20):
        ts = (now - datetime.timedelta(seconds=random.randint(0, 86400))).strftime("%b %d %H:%M:%S")
        f.write(f"{ts} kali sshd[1234]: Accepted publickey for daniel from 192.168.1.1 port {random.randint(40000,60000)} ssh2\n")
EOF
```

A. Write auth_analysis.py that reads auth.log and produces:

- A list of IPs that have more than 10 failed login attempts, sorted descending by attempt count
- The list of user accounts being targeted (which usernames are attackers guessing?)
- The ratio of failed to successful logins overall

B. Generate a synthetic web access log (or use a real one if available):

```
python3 - <<'EOF'
import random, datetime

ips = ["10.0.0.1", "185.220.101.5", "45.33.32.156", "66.249.66.1", "192.168.1.50"]
normal_paths = ["/", "/index.html", "/about", "/contact", "/static/main.css"]
attack_paths = [
    "/?id=1' UNION SELECT 1,2,3--",
    "/admin/../../../etc/passwd",
    "/search?q=<script>alert(1)</script>",
    "/wp-admin/",
    "/cgi-bin/test.cgi?cmd=id",
]
now = datetime.datetime.now()

with open("access.log", "w") as f:
    for hour in range(24):
```

```
        count = random.randint(80, 120)
        if hour == 3:  # Anomalous spike
            count = 950
        for _ in range(count):
            ip = random.choices(ips, weights=[30, 5, 5, 10, 3])[0]
            path = random.choices(normal_paths + attack_paths, weights=[20]*5 + [1]*5)[0]
            status = 200 if path in normal_paths else random.choice([200, 403, 500])
            ts = (now.replace(hour=hour, minute=random.randint(0,59))).strftime("%d/%b/%Y:%H:%M:%S +0000")
            f.write(f'{ip} - - [{ts}] "GET {path} HTTP/1.1" {status} {random.randint(200,5000)}\n')
EOF
```

C. Write log_analysis.py that reads access.log and produces:

- All requests matching SQL injection, path traversal, XSS, or command injection patterns (use the regex from the theory section as a starting point, but extend it)
- Top 5 IPs by request volume
- HTTP status code distribution

D. Implement the 3-sigma anomaly detector on hourly request counts. Your output should identify anomalous hours and report their z-score:

```
[ANOMALY] 03:00 — 950 requests (z=4.2σ, threshold=3.0σ)
```

E. Combine the outputs of A–D into a single report.md file with sections for each finding category.

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 auth_analysis.py --input auth.log

═══════════════════════════════════════════════════════
  AUTH LOG ANALYSIS — auth.log
═══════════════════════════════════════════════════════

[+] IPs with ≥ 10 failed attempts (brute-force suspects):
    185.220.101.5        268 attempts
    45.33.32.156         199 attempts
    10.0.0.2             22 attempts

[+] Targeted usernames:
    ubuntu           136 times
    daniel           124 times
    root             123 times
    admin            117 times

[+] Login summary:
    Failed logins  : 500
    Successful     : 20
    Fail/success   : 25.0:1
```

The auth log recorded 500 failed login attempts against 20 successful ones, yielding a fail-to-success ratio of 25:1 — a clear indicator of sustained automated brute-force activity. Three IPs exceeded the detection threshold of 10 failed attempts: 185.220.101.5 and 45.33.32.156 generated 268 and 199 attempts respectively, classifying them as high-risk external sources, while 10.0.0.2 contributed 22 attempts at medium risk. All four common usernames were targeted at roughly equal frequency (ubuntu, daniel, root, admin), which is consistent with dictionary-based credential stuffing rather than targeted attacks against known accounts.

```
Session  Actions  Edit  View  Help
└─$ python3 log_analysis.py --input access.log

═══════════════════════════════════════════════════════
  ACCESS LOG ANALYSIS — access.log
═══════════════════════════════════════════════════════

[+] Suspicious requests detected: 136
    [200] 66.249.66.1        /wp-admin/
    [403] 10.0.0.1           /admin/../../../etc/passwd
    [200] 185.220.101.5      /search?q=<script>alert(1)</script>
    [200] 10.0.0.1           /admin/../../../etc/passwd
    [403] 192.168.145.128    /admin/../../../etc/passwd
    [403] 10.0.0.1           /wp-admin/
    [200] 10.0.0.1           /search?q=<script>alert(1)</script>
    [403] 10.0.0.1           /cgi-bin/test.cgi?cmd=id
    [200] 10.0.0.1           /wp-admin/
    [403] 10.0.0.1           /cgi-bin/test.cgi?cmd=id
    ... and 126 more

[+] Top 5 IPs by request volume:
    10.0.0.1             1834 requests
    66.249.66.1          641 requests
    45.33.32.156         309 requests
    185.220.101.5        292 requests
    192.168.145.128      201 requests

[+] HTTP status code distribution:
    HTTP 200  →  3190
    HTTP 403  →  48
    HTTP 500  →  39

[+] Hourly anomaly detection (threshold=3.0σ):
    [ANOMALY] 03:00 — 946 requests (z=4.7σ, threshold=3.0σ)
```

The access log revealed 136 requests matching known attack signatures across four categories: path traversal (/admin/../../../etc/passwd), cross-site scripting (/search?q=<script>alert(1)</script>), command injection (/cgi-bin/test.cgi?cmd=id), and administrative panel probing (/wp-admin/). Of particular concern, several path traversal and XSS attempts returned HTTP 200, indicating the server processed the request rather than blocking it. By request volume, 10.0.0.1 dominated with 1,834 requests — over twice the second-highest source — suggesting a single aggressive scanner. The 3-sigma anomaly detector flagged hour 03:00 with 946 requests at z=4.7σ, nearly five standard deviations above the hourly mean, consistent with an automated scanning burst deliberately timed during low-traffic hours to reduce detection probability.

Question

The 3-sigma rule assumes data is approximately normally distributed. Web server traffic often has strong daily periodicity (high during business hours, low overnight). How does this periodicity affect the validity of a single global baseline? Describe a modified approach that would produce fewer false positives on a server with predictable daily traffic cycles.

How daily periodicity breaks the global baseline

The 3-sigma rule works well when data points are drawn from a single underlying distribution. Web traffic violates this assumption because it has two distinct behavioral regimes mixed together: high-traffic hours (business hours, typically 08:00–18:00) and low-traffic hours (overnight). When you compute a single global mean and standard deviation across all 24 hours, you are averaging across both regimes. The result is a baseline that accurately describes neither — it sits somewhere between the two, producing a standard deviation that is artificially inflated by the gap between day and night traffic levels.

This has two practical consequences. First, a legitimate business-hours spike that is perfectly normal for that time of day can appear anomalous against a global baseline that was dragged down by overnight quietness. Second, the inflated standard deviation raises the anomaly threshold, making it harder to detect genuinely suspicious overnight activity — exactly the window attackers prefer because traffic is sparse and anomalies blend in less.

In the lab data the effect is muted because the synthetic logs distribute traffic roughly uniformly across hours, with only the artificial hour-03 spike. On a real server with strong daily cycles, the global baseline would generate false positives during peak hours and miss real anomalies during quiet hours.

Modified approach: per-slot baseline with historical averaging

Instead of one global baseline, maintain a separate baseline for each hour-of-day slot, trained on the same hour across multiple days:

```
from collections import defaultdict
import statistics


def build_hourly_baselines(log_paths: list[str]) -> dict:
    """
    Build one baseline per hour-of-day from multiple days of logs.
    Returns {hour: (mean, stdev)} for each of the 24 slots.
    """
    hourly_samples = defaultdict(list)   # {"03": [950, 87, 102, ...], "14": [...]}


    for path in log_paths:
```

```
        day_counts = Counter()
        for line in Path(path).open():
            if m := LOG_PATTERN.match(line):
                day_counts[m.group("hour")] += 1
        for hour, count in day_counts.items():
            hourly_samples[hour].append(count)

    baselines = {}
    for hour, samples in hourly_samples.items():
        if len(samples) >= 2:
            baselines[hour] = (statistics.mean(samples), statistics.stdev(samples))
    return baselines



def detect_anomalies_per_slot(hourly_counts: Counter,
                              baselines: dict,
                              sigma_threshold: float = 3.0) -> list[str]:
    anomalies = []
    for hour, count in sorted(hourly_counts.items()):
        if hour not in baselines:
            continue
        mean, stdev = baselines[hour]
        if stdev == 0:
            continue
        z = (count - mean) / stdev
        if z > sigma_threshold:
```

```
        anomalies.append(
            f"[ANOMALY] {hour}:00 — {count} requests "
            f"(z={z:.1f}σ vs. {hour}:00 baseline of {mean:.0f} ± {stdev:.0f})"
        )
    return anomalies
```

This compares each hour only against the historical distribution for that same hour — 03:00 against past 03:00 counts, 14:00 against past 14:00 counts. A spike at 14:00 that is normal for a busy afternoon will not trigger an alert. An identical spike at 03:00, where the baseline is flat, will.

Why this produces fewer false positives

The standard deviation for each slot is computed within-regime rather than across regimes. Business-hour slots get a higher mean and a tighter band around it; overnight slots get a lower mean and a tighter band around their own quieter level. The anomaly threshold scales with the expected variability at that time, so both high-traffic and low-traffic hours are evaluated against an appropriate reference rather than a one-size-fits-all average. In practice, seven days of historical data per slot is enough to build a stable baseline for most servers.

Part 4: Integrated reconnaissance tool

Build recon.py — a single-file command-line tool that performs multi-stage reconnaissance on a domain or IP address and produces a structured report.

Requirements:

1. **CLI interface** via argparse:
    - target: domain name or IP address (required)
    - --mode: domain or ip, auto-detected if omitted
    - --output: directory for output files (default: ./recon_<target>_<timestamp>/)
    - --verbose: print progress to stderr as the tool runs
2. **Audit log** — every action the tool takes must be logged to audit.log inside the output directory with a timestamp. This includes: what command was run, what it returned (success/error), and the timestamp. This is non-negotiable.

3. **Domain mode** — when target is a domain name:
    - whois: extract registrar, registration date, expiry date, registrant organization
    - dig: query and save A, MX, NS, and TXT records
    - curl -I: extract HTTP response headers (server, X-Powered-By, Content-Security-Policy, Strict-Transport-Security)
    - Each step runs independently; a failure in one must not stop the others
4. **IP mode** — when target is an IP address:
    - nmap -sV --open -oX: scan common ports (use the --top-ports 100 flag to keep it fast), parse the XML output as in Part 2
    - Reverse DNS lookup via dig -x
    - whois on the IP to extract the owning organization and country
5. **Structured output** — write results.json with all findings in a single dict keyed by tool name.
6. **Markdown report** — generate report.md from results.json. The report must include: a summary table of findings, open ports (if IP mode), DNS records (if domain mode), and any notable security headers that are missing (CSP, HSTS, X-Frame-Options).

Grading criteria:

Criterion	Weight
Audit log present and accurate	20%
Each step fails independently without crashing	20%
results.json contains structured data (not raw text)	25%
report.md is readable and complete	20%
Argparse works correctly for both modes	15%

With IP address:

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 recon.py 192.168.145.128 --verbose
2026-05-09 10:22:31,490 [INFO] ═══ recon.py started | target=192.168.145.128 mode=ip output=recon_192.168.145.128_20260509_102231 ═══
[*] Mode: ip | Target: 192.168.145.128
[*] Running nmap ...
2026-05-09 10:22:31,491 [INFO] RUN: nmap -sV --open --top-ports 100 -oX recon_192.168.145.128_20260509_102231/nmap.xml 192.168.145.128
2026-05-09 10:22:40,434 [INFO] OK:  nmap -sV --open --top-ports 100 -oX recon_192.168.145.128_20260509_102231/nmap.xml 192.168.145.128 (exi
t 0)
[*] Running reverse DNS lookup ...
2026-05-09 10:22:40,440 [INFO] RUN: dig -x 192.168.145.128 +short
2026-05-09 10:22:40,512 [INFO] OK:  dig -x 192.168.145.128 +short (exit 0)
[*] Running whois ...
2026-05-09 10:22:40,512 [INFO] RUN: whois 192.168.145.128
2026-05-09 10:23:00,536 [ERROR] TIMEOUT: whois 192.168.145.128 after 20s
[*] Writing results.json and report.md ...
2026-05-09 10:23:00,539 [INFO] ═══ recon.py finished | output=recon_192.168.145.128_20260509_102231 ═══

[√] Done — output in: recon_192.168.145.128_20260509_102231/
    ├── results.json
    ├── report.md
    ├── audit.log
    └── nmap.xml
```

With domain name:

```
┌──(kali㉿kali)-[~/lab-auto]
└─$ python3 recon.py google.com --verbose
2026-05-09 10:23:30,207 [INFO] ═══ recon.py started | target=google.com mode=domain output=recon_google.com_20260509_102330 ═══
[*] Mode: domain | Target: google.com
[*] Running whois ...
2026-05-09 10:23:30,207 [INFO] RUN: whois google.com
2026-05-09 10:23:50,216 [ERROR] TIMEOUT: whois google.com after 20s
[*] Running dig (A, MX, NS, TXT) ...
2026-05-09 10:23:50,217 [INFO] RUN: dig +short A google.com
2026-05-09 10:23:50,276 [INFO] OK:  dig +short A google.com (exit 0)
2026-05-09 10:23:50,276 [INFO] RUN: dig +short MX google.com
2026-05-09 10:23:50,366 [INFO] OK:  dig +short MX google.com (exit 0)
2026-05-09 10:23:50,367 [INFO] RUN: dig +short NS google.com
2026-05-09 10:23:50,470 [INFO] OK:  dig +short NS google.com (exit 0)
2026-05-09 10:23:50,470 [INFO] RUN: dig +short TXT google.com
2026-05-09 10:23:50,568 [INFO] OK:  dig +short TXT google.com (exit 0)
[*] Running curl -I ...
2026-05-09 10:23:50,569 [INFO] RUN: curl -I -s --max-time 10 https://google.com
2026-05-09 10:23:51,016 [INFO] OK:  curl -I -s --max-time 10 https://google.com (exit 0)
[*] Writing results.json and report.md ...
2026-05-09 10:23:51,016 [INFO] ═══ recon.py finished | output=recon_google.com_20260509_102330 ═══

[√] Done — output in: recon_google.com_20260509_102330/
    ├── results.json
    ├── report.md
    ├── audit.log
```

Question

Your tool performs **active reconnaissance**: it sends packets to the target. Shodan performs **passive reconnaissance**: it has already scanned the internet, and you query its database without touching the target at all. From the perspective of an attacker, what are the operational differences between these two approaches? From the perspective of a defender with network monitoring in place, which approach is harder to detect, and why? In what scenarios would each be more appropriate?

Operational differences for an attacker

Active reconnaissance — as our recon.py does — means the attacker's machine sends packets directly to the target. The tool establishes real TCP connections, sends DNS queries, makes HTTP requests, and runs nmap probes. All of this generates traffic that originates from the attacker's IP address and arrives at the target's network perimeter. The attacker gets fresh, real-time data: exactly what ports are open right now, what version is running today, what headers the server returns at this moment.

Passive reconnaissance via Shodan means the attacker never touches the target at all. They query a database that was populated by Shodan's own internet-wide scanners days or weeks ago. The attacker's IP appears only in Shodan's query logs, never in the target's logs. The tradeoff is data freshness — a service that was exposed last month may have been patched or taken down since the last Shodan crawl.

Which is harder to detect for a defender

Passive reconnaissance is essentially undetectable at the network level. No packet from the attacker ever crosses the target's perimeter, so no firewall, IDS, or packet capture will record it. The only entity that could detect it is Shodan itself, and they do not share query logs with targets. A defender with full network monitoring in place — even with a SIEM, NetFlow analysis, and an IDS tuned for port scans — will see zero evidence that someone queried Shodan for their IP range.

Active reconnaissance is detectable by design. Your recon.py leaves traces in multiple places simultaneously: nmap SYN packets appear in firewall logs and trigger IDS signatures, the curl -IL request appears in the web server's access.log with the attacker's IP and user-agent, and the dig queries may appear in DNS resolver logs. A defender running something equivalent to the detect_brute_force() and detect_traffic_spikes() functions you built in Part 3 would flag an nmap scan almost immediately — it is one of the oldest and most reliable IDS signatures in existence.

When each approach is more appropriate

Passive reconnaissance is appropriate in the early intelligence-gathering phase when the attacker's priority is staying undetected and building a target profile without any risk of triggering alerts. It is also useful for broad target selection — an attacker can query Shodan for thousands of IPs matching a vulnerability signature without ever touching any of them, then select the most promising targets for active follow-up.

Active reconnaissance becomes necessary when the attacker needs information Shodan cannot provide: current service state, real-time header values, certificates that rotated after the last crawl, or internal network ranges that are not internet-facing and therefore never indexed by Shodan. It is also required when the attacker needs to interact with the target to confirm a vulnerability rather than just observe its existence.

In practice, sophisticated attackers chain both: passive first to avoid any footprint during target selection, active only against the final target and timed during high-traffic windows — exactly the behavior the 3-sigma detector in Part 3 was designed to catch, and exactly why it flagged the anomalous hour-03 spike in your synthetic logs.